A Certificate-Based Branching Analysis for MaxSAT Parameterized by the Number of Clauses

Abstract

We present a pipeline for constructing and checking local branching analyses for exact MaxSAT. Starting from a partially specified clause neighborhood, the generator distinguishes empty and nonempty clause tails, exposes further variables, and searches for branching strategies. It combines assignment enumeration, removal of dominated alternatives, grouping by variable substitutions, reductions, and recursive analysis of the resulting formulas. The output is a certificate recording both the coverage of the case analysis and the transformations used in each case. A separate checker reconstructs the refinements, checks local optimum identities, and evaluates branching inequalities with exact rational arithmetic.

We apply the pipeline to the $(3, 2)$-literal bottleneck in Xiao's $O^*(1.2886^m)$-time algorithm, where m is the number of clauses. Combining generated strategies with branching lemmas from Xiao, the local analysis reaches the target $q = 1.2872$. We prove the soundness of the certificate rules and state the conditions under which this local result yields an $O^*(1.2872^m)$-time algorithm. The contribution is a compositional, machine-checkable analysis pipeline relative to a fixed library of imported branching lemmas, with the improved local target serving as its case study.

1 Introduction

A good branching rule for an exact algorithm must do two things: preserve an optimal solution among its children and reduce the instance enough to meet the desired running-time bound. Both tasks become harder when a useful rule requires several rounds of branching and reduction. The argument must also cover every possible local configuration, including configurations whose clauses contain variables outside the displayed neighborhood.

We study how to automate this analysis for MaxSAT, using the number of clauses as the measure. Our starting point is Xiao's $O^*(1.2886^m)$-time, polynomial-space algorithm [1]. Its limiting case concerns a variable occurring three times positively and twice negatively. Replacing this case by a better local strategy offers a concrete test of an automated analysis tool: the target is known, the relevant neighborhood is small, and the surrounding algorithm supplies structural information that can guide the search.

Our pipeline separates two trees that play different roles. A *refinement tree* distinguishes possible clause neighborhoods. For each of its terminal cases, a *strategy tree* describes how to reduce or branch on an instance in that case. The generator builds these trees from an analysis script. Within each selected case, it enumerates assignments, removes dominated choices, groups compatible assignments, and analyzes the residual formulas. A separate checker reads the resulting trees without repeating this search.

The key verification step is simple. For each possible truth-value combination of the unknown clause tails, the checker computes the best local score before and after a proposed transformation. The scores must agree after accounting for the clauses already satisfied. This single test handles reductions and grouped branches through the same interface. Recurrence bounds are checked separately from these objective-value identities.

Contributions.

1. We give a pipeline that combines structural refinement with recursive search for local MaxSAT strategies, including assignment dominance, substitution-based grouping, and reductions of the resulting children.
2. We define certificate rules for coverage, local transformations, and accumulated measure decreases, and prove their compositional soundness.
3. We apply the pipeline at the rational target $q = 1.2872$ and give an explicit successful strategy from the generated analysis. We separate the generated transformations from the imported results used to derive the running-time bound.

1.1 Related work and positioning

Automatic branching analysis has a substantial history. Fedin and Kulikov [2] generated running-time proofs from supplied simplification rules and reported experiments for SAT, MaxSAT, and bounded-occurrence MaxSAT. Kulikov [3] also automated the discovery of simplification rules and useful splittings for SAT and MaxSAT. Skjernaa's framework [4, Chapter 8] refines formula patterns, searches for reductions, and considers branching on variable values and on whether pairs of variables are equal or different. Thus automatic case analysis, reduction discovery, and branching beyond individual assignments are established ideas.

More recently, Červený and Suchý [5] generated parameterized branching algorithms for d-Path Vertex Cover. Gao et al. [6] formulate local branching-rule selection for Maximum Independent Set as a weighted set-cover problem. These works also motivate searching for a collection of alternatives rather than choosing a single branching variable.

Our focus is the integration of clause-tail refinements, recursive analysis of substitution-generated residual CNFs, and a separate checker for both coverage and local optimum identities. In particular, the certificate records actual child formulas and their composition, so a newly added search heuristic does not require a corresponding heuristic-specific verification rule. We present this combination for Xiao's bottleneck; we do not claim that deeper search, dominance, or grouping alone is new.

The underlying MaxSAT algorithm builds on the clause-measure analyses of Bansal and Raman [7], Chen and Kanj [8], and Xu et al. [9]; related resolution techniques appear in Chen, Xu, and Wang [10]. Other parameterizations, including those studied by Bliznets and Golovnev [11], lead to different bounds. General background on branching recurrences is provided by Fomin and Kratsch [12].

Finally, proof logging for MaxSAT can certify optimum-preserving preprocessing, including through the VeriPB and CakePB systems [13]. Our certificate has a different target: a local case analysis and its recurrence for a family of instances. Its semantic checks enumerate small local truth tables, while pseudo-Boolean proof logging supports reasoning about individual, potentially large solver instances.

2 Preliminaries and the target case

We consider unweighted MaxSAT. A formula F is a multiset of clauses, and a clause is a disjunction of literals. Equal clauses are retained with multiplicity. Duplicate literals within a clause can be removed; tautological clauses can be removed with an objective offset of one each, and empty clauses with offset zero. We assume this normalization at the start of the algorithm. Write $m = |F|$ for the resulting number of clauses and

$$\mathrm{OPT}(F) = \max_{\alpha} \mathrm{sat}(F, \alpha),$$

where α ranges over all assignments and sat counts satisfied clauses with multiplicity. We describe transformations of the optimum directly; equivalently, the decision problem asks whether $\mathrm{OPT}(F) \geq k$.

A literal x has degree (i, j) if it occurs in i clauses and its complement $\bar{x}$ in j clauses. The complementary literal has degree (j, i). Following Xiao, an $(i, 1)$-literal is a *singleton* when its unique negative occurrence is the unit clause $(\bar{x})$.

2.1 Scores and measure decreases

A correct branching step produces formulas F_i and nonnegative integer *offsets* s_i such that

$$\mathrm{OPT}(F) = \max_i \big(s_i + \mathrm{OPT}(F_i)\big).$$

Its *measure decreases* are $d_i = |F| - |F_i|$. These need not equal the offsets: an empty false clause contributes to d_i but not to s_i. For example, assigning $x = 1$ in $(x)(\bar{x})$ gives $s = 1$ and $d = 2$. Juxtaposition of clauses denotes their conjunction, with each occurrence counted separately.

For a rational target $q > 1$, a vector of positive integer decreases is *q-admissible* when

$$\sum_{i=1}^{r} q^{-d_i} \leq 1. \tag{1}$$

We use $q = 12872/10000 = 1.2872$ and check this inequality exactly. The notation O^* suppresses factors polynomial in the input length.

Lemma 1 (Branching recurrence)**.** *Suppose each recursive step has positive decreases satisfying equation (1), and the work at each node, including any intervening reductions, is polynomial in the input length. Then the running time is $O^*(q^m)$.*

Proof. Induction on m bounds the number of leaves by q^m: an internal node has at most $\sum_i q^{m-d_i} \leq q^m$ descendant leaves. Each root-to-leaf path has at most m recursive edges, since every decrease is positive. The number of nodes is therefore at most $(m + 1)q^m$. Polynomial work per node gives the claimed bound. □

2.2 The $(3, 2)$-literal bottleneck

Xiao's algorithm applies its reductions and branching cases in a fixed order. Step 5 handles a $(3, 2)$-literal x, whose incident clauses are

$$(x \vee C_1)(x \vee C_2)(x \vee C_3)(\bar{x} \vee D_1)(\bar{x} \vee D_2). \tag{2}$$

Step 5.4, where D_1 and D_2 contain only degree-five variables, gives the limiting vector $(6, 6, 5, 5)$ [1, Section 4]. Other cases motivate the chosen target: for example, $(11, 3, 3)$ has branching factor approximately 1.287190881, just below 1.2872.

The generator begins with the five-clause pattern in equation (2), allowing each displayed tail to be empty. It can therefore encounter cases covered by neighboring steps as well as the intended bottleneck. The admissible degrees of newly exposed literals are part of the analysis configuration. The exact root pattern and the degree list used by `solve.txt` are given in section 6.1; their connection to Xiao's ordered algorithm is separated from the local checking argument in section 6.

3 Pipeline overview and a worked example

A *template* displays selected variables and uses ? for the remaining part of a clause. The symbol $?^+$ means that this part contains at least one literal. These marks describe syntax: $?^+$ can still be false under an assignment. Subscripts identify different tails when their identities matter. All occurrences of a selected variable are displayed, including occurrences outside the original five clauses.

The refinement tree first asks structural questions. Is a chosen tail empty? If not, which degree can a literal in it have, and where can its other occurrences lie? Each answer gives a child template. The strategy search then asks an optimization question: which assignments or groups of assignments cover an optimal solution with a sufficiently good branching vector? Figure 1 summarizes the two stages.

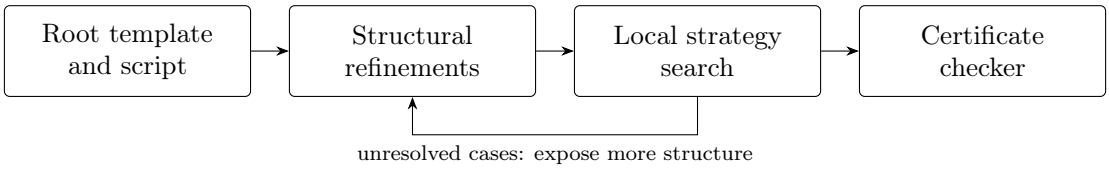


Figure 1: The script selects refinements and search budgets. The generator constructs local strategies; the checker validates the resulting certificate.

Example 1 (A grouped branch from the certificate)**.** Consider the following template, obtained by exposing a $(3,1)$-singleton y in the two negative clauses of the root:

$$\begin{aligned} P = {} & (x \vee ?_1)(x \vee ?_2)(x \vee ?_3) \\ & (\bar{x} \vee y \vee ?_4)(\bar{x} \vee y \vee ?_5)(y \vee ?_6)(\bar{y}). \end{aligned} \tag{3}$$

The four assignments to (x, y) need only two alternatives. The assignments $(0,0)$ and $(1,0)$ are grouped by fixing $y = 0$ and leaving x free. When $y = 1$, changing x from 0 to 1 cannot lose a satisfied clause: the two negative x-clauses are already satisfied by y. Thus $(0,1)$ is dominated by $(1,1)$.

For the first alternative, let

$$H = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5)(?_6).$$

The unit clause $(\bar{y})$ contributes offset 1, and all six clauses of H remain in the recursive formula, including the tail-only clause $(?_6)$. The second alternative satisfies six clauses and falsifies $(\bar{y})$, leaving no local clause. It has offset 6 and decrease 7. Consequently, for every choice of tail values,

$$\mathrm{OPT}(P) = \max\{1 + \mathrm{OPT}(H),\ 6\}. \tag{4}$$

Here and in this example only, the fixed tail values are implicit in OPT. The vector is $(1,7)$, and $q^{-1} + q^{-7} < 0.948 < 1$ at $q = 1.2872$.

This example also explains why the generator constructs residual formulas. A grouped alternative can retain a free variable, so further reductions and branching can improve its contribution to the final vector. The search can continue below H instead of treating the group as an indivisible leaf.

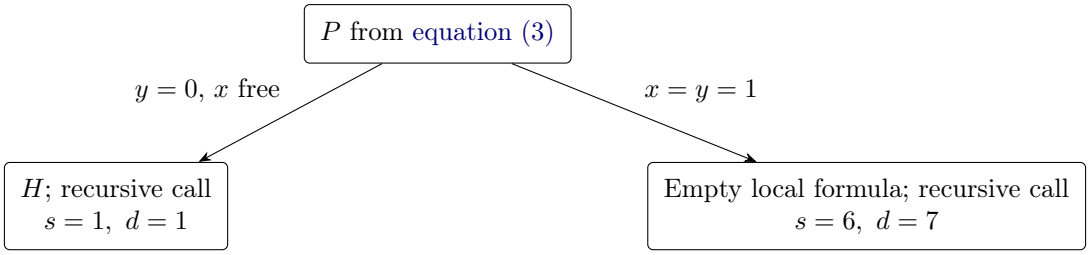


Figure 2: A strategy extracted from the existing certificate, with variable and tail names relabeled. The external part of the instance remains to be solved even when the local formula is empty.

4 Templates and certificate rules

We now formalize the two checks used above: refinements cover the relevant instances, and strategies preserve their optimum. We use F for a complete recursive formula and P for its displayed template.

4.1 Unknown tails and the surrounding formula

A concrete instance matches P if it can be written as $F = P[T] \uplus E$. Here $P[T]$ replaces each tail by a disjunction of literals on unselected variables, and E is the multiset of remaining clauses. No selected variable occurs in a tail or in E. Repeated tail identifiers denote the same disjunction. A $?^+$ tail must be nonempty; a ? tail may be empty. Matching also respects any degree and singleton restrictions used by the analysis.

Within a strategy, tail identities are preserved across transformations: a child may omit a tail but cannot introduce an unrelated one. Structural refinement has a different role. When a variable is exposed from a tail, the child's tail describes the remainder after that exposure, and the displayed fragment may acquire further clauses from E. Matching is always interpreted on the same complete instance, even though its displayed fragment changes.

For a Boolean vector τ indexed by the tails, define the local score

$$B_P(\tau) = \max_{\alpha} \operatorname{sat}(P, \alpha, \tau), \tag{5}$$

where α assigns the selected variables. Tails are evaluated as Boolean inputs; a clause made true by a tail still counts as one satisfied clause. Different tails may share hidden variables in a concrete instance. Checking all Boolean vectors τ includes every realizable combination and may therefore reject some otherwise valid transformations, but cannot accept an invalid one for this reason.

4.2 Structural refinements

Refinements distinguish formula structure without making a Boolean assignment. The two basic operations are shown in figure 3.

Tail split. Replace a chosen ? by the two cases "empty" and $?^+$. These cover every concrete tail. This is recorded as `tail_empty_or_nonempty`.

Variable exposure. Choose a literal from a nonempty tail and name it y. For each allowed oriented degree (i, j), enumerate all placements of y and $\bar{y}$ in existing clause tails, and add clauses for the remaining occurrences. The selected clause must contain y; singleton restrictions fix the unique negative occurrence to a unit clause. The remaining tail in each affected clause may be

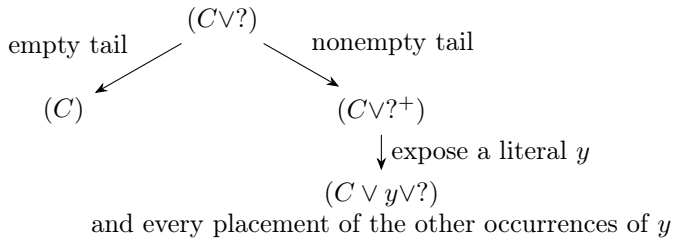


Figure 3: Topological refinement nodes. If C is nonempty, the left child is an ordinary clause, not an empty clause. After exposing one literal from $?^{+}$, the remaining tail is ? and may be empty.

empty. Each concrete occurrence pattern is among these choices, provided the degree list covers the literals that can be selected.

The format combines a tail split and these exposures in `expose_at_clause`: an empty child is included for ? and omitted for $?^{+}$. Exposure is restricted to templates with distinct, clause-local single tails. An initial variable declaration constructs the root family; it is not a refinement of arbitrary formulas.

The coverage requirement can be stated without a separate semantics for fragments: *every complete instance matching the parent and its structural restrictions must match at least one child.* Exposing y merely moves all its occurrences into the displayed fragment of that same instance. The checker regenerates the child templates, compares them up to renaming, and rejects missing, extra, or duplicate cases. In format version 2 the allowed degree list is recorded in the exposure node. Its completeness under the intended structural restrictions is a mathematical premise, whereas faithful enumeration of its incidences is checked computationally.

4.3 Local strategies

The certificate has four operations: refinement, decomposition, recursive call, and imported lemma application. Decompositions have two forms.

Assignment decomposition. The alternatives assign a selected variable x to 0 and 1. The checker simplifies both children and recomputes their offsets and decreases. Suppose an assignment leaves a displayed fragment P_i and transfers a multiset R_i of tail-only clauses to the surrounding formula. Then

$$F_i = P_i[T] \uplus E \uplus R_i[T], \qquad d_i = |P| - |P_i| - |R_i|. \tag{6}$$

Satisfied clauses contribute to s_i; empty false clauses contribute only to d_i. The transferred clauses remain part of the recursive instance. For example, setting $x = 0$ in $(x\vee?)(\bar{x})$ transfers $(?)$ and gives $s = d = 1$, even though no selected variable remains.

Semantic decomposition. The certificate supplies children $H_1, \ldots, H_r$ and offsets s_i. The checker accepts the transformation when

$$B_P(\tau) = \max_{1\le i\le r} \big(s_i + B_{H_i}(\tau)\big) \quad \text{for every } \tau. \tag{7}$$

Every child uses only parent tails and selected or fresh local variables. Here the surrounding formula is unchanged, so tail-only clauses that remain must be included in H_i, as in example 1. The decrease is $d_i = |P| - |H_i|$. A one-child decomposition describes a reduction; its offset can also be inferred if $B_P - B_{H_1}$ is a constant nonnegative integer.

Lemma 2 (Replacement inside a formula)**.** *Suppose equation (7) holds and the local variables of the parent and children occur nowhere in the surrounding formula or the concrete tails. Then replacing $P[T]$ by the alternatives $(H_i[T], s_i)$ preserves the optimum of $P[T] \uplus E$.*

Proof. Fix an assignment β to all external variables. It gives tail values τ_β and satisfies some $e(\beta)$ clauses in E. The parent optimum is

$$\max_\beta \big(e(\beta) + B_P(\tau_\beta)\big) = \max_i \left(s_i + \max_\beta \big(e(\beta) + B_{H_i}(\tau_\beta)\big) \right),$$

using equation (7). The expression on the right is the best shifted optimum of a child in the same surrounding formula. □

Recursive call and imported lemma. A `call` ends the local analysis and invokes the global algorithm on the remaining complete formula. It adds zero to the decrease already accumulated on its path. An imported lemma application, encoded as `assumption`, supplies a named branching vector instead. The checker records its name, formula, vector, and tree path; its mathematical justification comes from the cited branching result and its applicability to the current instance. The imported results are Xiao's Lemmas 2–4 and Step 5.1, together with the repeated Lemma 3 argument in Step 5.5. Table 1 gives the correspondence to the certificate names.

Certificate name	Mathematical dependency
`lemma2`	Xiao's Lemma 2 for an $(i, 1)$ non-singleton.
`lemma3`	Xiao's Lemma 3 for a degree-three variable.
`lemma4`	Xiao's Lemma 4 for specified unit occurrences.
`double_lemma3`	The composition of Lemma 3 used in Step 5.5, with the second singleton preserved in the resolution child.
`step5.1`	The unit-clause case for a $(3, 2)$-literal.

Table 1: Imported branching arguments, with source names retained for tracing certificate records to Xiao [1]. Each use requires the source hypotheses, including any reduction-or-branch alternative and the claimed decreases in the complete instance. The checker logs these uses rather than reconstructing their children or verifying their applicability.

Why Lemma 3 can be used twice. Consider the branching case of Lemma 3 for a $(2, 1)$-singleton y_1. Its resolution child replaces

$$(y_1 \vee A)(y_1 \vee B)(\bar{y}_1) \quad \text{by} \quad (A)(B)$$

with offset and decrease one. In this case A and B are nonempty; short clauses that trigger the source reductions are handled before this branching case. Removing y_1 from these two clauses preserves every occurrence of any other variable, since clause multiplicities are retained. In particular, a distinct $(2, 1)$-singleton y_2 retains its two positive occurrences and its negative unit clause. When the branching case of Lemma 3 applies again, the two nested vectors $(1, 8)$ give

$$(8,\ 1 + 8,\ 1 + 1) = (8, 9, 2).$$

This is the composition used in Xiao's Step 5.5. The reduction alternatives of the imported lemmas must still be handled as reductions when they apply; the calculation above describes their nested branching case.

4.4 Composition and soundness

A decomposition edge with decrease d followed by a child vector $(e_1, \ldots, e_t)$ contributes $(d + e_1, \ldots, d + e_t)$ to the enclosing strategy. Concatenating the vectors of all alternatives gives its leaf vector. Only this accumulated vector must have positive entries and satisfy equation (1); an intermediate reduction may have decrease zero. Refinement children are separate structural cases, so their vectors are checked separately, not concatenated.

Theorem 3 (Local certificate soundness)**.** *Let a finite certificate pass the refinement, decomposition, and vector checks above at target q. Suppose its exposure degree lists are complete under the stated structural restrictions, and each recorded imported lemma admits a correct recursive strategy with at least the recorded decreases for every complete instance to which it is applied. Then every such instance matching the root has an optimum-preserving local strategy whose accumulated decreases are q-admissible.*

Proof. Induct over the certificate. A tail split covers its parent, and variable exposure enumerates every permitted incidence pattern. Thus at least one refinement child applies to each matching instance. Assignment decomposition is exhaustive, with the clause accounting in equation (6). Semantic decomposition preserves the optimum by lemma 2. Calls end a local path, and imported applications use the stated hypothesis. Edge decreases telescope along each path; larger actual decreases can only improve equation (1). The check at each strategy root establishes the claimed recurrence. □

5 Automated search and implementation

The C++ generator maintains a frontier of unresolved templates. The analysis script chooses which tails to expose and when to run a search pass. Each pass first tries a cheap enumeration-based strategy, then applies the requested stronger analysis to the remaining templates. Successful cases are removed from the frontier and their proof trees are logged. This division lets a short script direct the structural analysis while the program searches the local strategies automatically.

5.1 Dominance, grouping, and recursive search

For a selected set of variables, the generator enumerates assignments and records which clauses are satisfied, which become empty, and which remain unresolved. It removes an assignment if another choice is at least as good for every possible tail valuation. The implementation uses clause masks for these comparisons and records them as *subsumptions*. This is dominance among assignment alternatives, not unrestricted deletion of subsumed CNF clauses in MaxSAT.

The surviving assignments are candidates for grouping. A group is represented by substituting each selected variable with a constant or a signed fresh variable:

$$x_j \longmapsto 0,\ 1,\ u_1,\ \bar{u}_1, \ldots, u_t, \bar{u}_t. \tag{8}$$

For example, $\{(0,0),(1,0)\}$ is represented by $(x,y) = (u,0)$, and $\{(0,1),(1,0)\}$ by $(x,y) = (u,\bar{u})$. Substitution produces an ordinary residual CNF to which reductions and further branching can be applied. The group must be exactly the image of the substitution; arbitrary affine constraints such as $x_3 = u_1 \oplus u_2$ are outside this representation.

The recursive variable search tries each available pivot and analyzes both assignment children up to the requested depth. For each child it may either stop with a call or retain its deeper analysis. Thus a useful strategy can be asymmetric. Grouped residuals also receive reductions and child analysis; the group pass includes an ordinary one-level variable search as a baseline. The depth parameters bound the search for an assignment tree, not the depth of the final global algorithm.

A candidate is evaluated by its accumulated leaf vector. Floating-point branching factors guide the C++ search; certificate acceptance uses the exact rational target. Reductions that remove clauses can end in a recursive call. The implemented zero-clause-decrease reductions eliminate a variable before continuing locally. Assignment depth limits and these decreases make each such local search finite. The outer refinement schedule is supplied by the script; the pipeline makes no completeness claim about the search for a strategy at an arbitrary smaller target.

5.2 Certificate construction and checking

The generator logs the chosen refinements and strategies in JSON Lines. An assembler connects these records into one tree and rejects unresolved cases. The resulting JSON certificate is read by a separate Python checker. Formula normalization, refinement enumeration, assignment simplification, optimum tables, and rational vector arithmetic are implemented independently of the C++ search. Four certificate operations suffice because proposed reductions, dominance eliminations, and grouped branches share equation (7).

For a template with a local variables, t tails, and representation length $\ell(P)$, a straightforward optimum-table computation takes $O(2^{a+t}\ell(P))$ time. This exponential work is confined to the finite local patterns used during offline verification. Once a certificate is fixed, its tables and tree sizes are constants with respect to the size of a future input instance.

The checker uses NumPy integer arrays for the score tables. It enforces a limit of 32767 clauses per local formula, stores scores in signed 16-bit arrays, and performs validated offset additions and differences in signed 32-bit arrays. Branching inequalities use exact rational arithmetic. The trusted implementation comprises the checker and its Python, JSON, and NumPy runtime; it is not a formally verified implementation of the calculus.

6 The generated analysis and its global consequence

6.1 Root family and analysis script

The root declared by `solve.txt` is

$$P_0 = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5). \tag{9}$$

All five tails are initially arbitrary. The default exposure classes in this session are

$$(3,1)\text{-singleton}, \quad (2,2), \quad (3,2), \quad (2,3), \quad (4,1)\text{-singleton}.$$

Degrees are oriented toward the literal selected in the clause. The session first exposes y in a clause containing $\bar{x}$. It then exposes z in a clause containing $\bar{x}$ but not y, interleaving these refinements with strategy searches. The commands are reproduced in figure 4; blank lines after `addpos` are part of its clause-selection input.

The completed local analysis reaches $q = 1.2872$ using the imported results in table 1. Example 1 gives one of its explicit strategies, with no imported application in that strategy. For a complete certificate, every refinement leaf must have a strategy passing the local vector check. This is the computational result to which theorem 3 applies.

Bounded-width mode. With `-max-clause-size` k, the generated certificate proves only the declared local template family. The degree list stored at every variable exposure is an explicit premise of that family; enumerating placements proves coverage relative to this list, not completeness of the list for arbitrary k-MaxSAT. Clause width does not bound variable degree, and the 30-bit mask limit is only an implementation limit of the local search. A global k-MaxSAT bound would additionally require an ordered treatment of every omitted degree and structural case, a width-preserving recursive algorithm, its terminal algorithm, and the corresponding

```
set 1.2872

add x 3 2 a
addpos y
-x

branch
addpos z
-x
y
xiao_branch 1
xiao_branch 2
branch
empty_divide -1
branch

print 0
exit
```

`add` declares the root; `a` allows arbitrary tails.

`addpos` reads a name, required clause literals, and excluded clause literals. It distinguishes empty tails and enumerates variable exposures.

`branch` runs enumeration and grouping. `xiao_branch 1` and `xiao_branch 2` request recursive variable searches of depths one and two.

`empty_divide -1` refines eligible tails in the unresolved frontier. The last `branch` searches the resulting cases.

Figure 4: The analysis session in `solve.txt`. The script controls the refinement schedule; local strategies are selected by the generator.

recurrences. Imported-lemma leaves are disabled by the generator and rejected by the checker in bounded-width mode, so each accepted bounded certificate contains only explicit checked transitions.

6.2 From a local strategy to a running-time bound

We retain Xiao's reductions and all ordered cases except Step 5.4. Their connection to the generated analysis can be made explicit.

Coverage. At the start of Step 5, Xiao's preceding reductions and Steps 1–4 leave only $(2,2)$-variables, $(3,2)$-variables, and $(3,1)$- or $(4,1)$-singletons [1, Section 4.6]. A singleton's negative literal occurs only in its own unit clause, so it cannot be a hidden literal in a clause with a selected variable. Orienting a newly exposed literal toward the selected clause gives exactly the five classes in section 6.1. The initial five occurrences of x are precisely equation (9). Thus every instance reaching Step 5.4 matches the root under these restrictions. Structural refinements only reveal the same instance, so the degree restrictions remain valid throughout that refinement tree.

The other recurrences. Table 2 checks the distinct vectors outside Step 5.4 in Xiao's global case table. Every check uses rational arithmetic; the displayed numbers are upper bounds rounded upward to six decimal places. In particular, both Steps 5.2 and 6.3 use $(11,3,3)$ and satisfy the target. The terminal Step 7 takes $O^*(1.2576^m)$ time by Xiao's Lemma 5, which is within the same bound.

Using a fixed certificate. The completed certificate is fixed independently of the input size. A refinement inspects whether a concrete tail is empty or selects a literal from it, scans its occurrences, and follows a matching child. Even enumerating all mappings of a fixed number of displayed variables takes polynomial time in the input length. The recorded assignments and substitutions are performed on the concrete clauses, with fresh local variables chosen outside the instance. Tail-only clauses remain in the recursive formula as in equation (6). A strategy has fixed finite depth and ends in calls whose accumulated decreases are positive. Imported routines

Xiao's steps	Branching vector	Upper bound on $\sum_i q^{-d_i}$
1, 5.3, 6.2	$(6, 1)$	0.996729
2	$(3, 3)$	0.937761
3	$(8, 1)$	0.909569
4	$(5, 2)$	0.886532
5.1, 6.1	$(4, 2)$	0.967807
5.2, 6.3	$(11, 3, 3)$	0.999976
5.5	$(11, 10, 4, 3)$	0.975442
6.4	$(18, 18, 11, 11, 11, 11, 4, 4)$	0.998639

Table 2: The unchanged cases at $q = 12872/10000$. Vectors are taken from Xiao [1, Table 3]; the rational checks isolate Step 5.4 as the case to replace.

use the polynomial operations of the source algorithm. These facts give polynomial time and space for local matching, transformations, and dispatch, while all search for the certificate is performed offline.

Theorem 4 (Global consequence)**.** *Suppose the completed certificate at $q = 12872/10000$ satisfies the premises of theorem 3, with the imported results applied under their source hypotheses. Replacing Xiao's Step 5.4 by its recorded strategies solves* MaxSAT *in $O^*(1.2872^m)$ time and polynomial space.*

Proof. Follow Xiao's ordered algorithm, using the recorded local strategy when the replaced case is reached. Coverage permits this choice, and theorem 3 gives its correctness and branching inequality. The remaining cases satisfy the same target by table 2 and the bound for Step 7. The fixed-certificate dispatch described above and lemma 1 bound the branching work. Polynomial work at a leaf may be replaced by the Step 7 subroutine: a leaf of remaining measure m' has cost $O^*(1.2576^{m'}) \leq O^*(q^{m'})$, and the branching inequalities bound the sum of these leaf weights by q^m. A depth-first traversal has polynomial depth and stores polynomial information at each level. Together with the polynomial-space terminal subroutine, this proves the space bound. □

The imported lemmas are mathematical results used by this algorithm. Their certificate encoding determines the extent of machine checking: explicit transitions and recurrence arithmetic are checked, whereas imported applications are logged for inspection under their source hypotheses.

6.3 Reproducing the analysis

The generation session is run from the repository root after building the C++ executable. The following commands generate, assemble, and check the certificate; the final command can also be run on an existing certificate without repeating generation.

```
make
./maxsat_prover test < solve.txt
python3 stitch_certificate.py pre_certificate.jsonl proof_tree.json \
  --numerator 12872 --denominator 10000
python3 verify_certificate.py proof_tree.json imported_lemmas.json
```

Generation recreates `pre_certificate.jsonl`. Verification writes the imported applications to `imported_lemmas.json`. Its verdict is `ACCEPT` when this log is empty; otherwise it reports

```
ACCEPT RELATIVE TO IMPORTED LEMMAS.
```

The artifact accompanying this version was generated at target 12872/10000 and assembled as a version-2 certificate. Its structure and the measured costs of assembly and checking are summarized in table 3. Times are wall-clock times from one run on an AMD Ryzen 7 5800U

machine with 16 hardware threads and 15 GiB of memory, using Python 3.10.6, NumPy 2.2.6, and GCC 15.2.1. Generation was performed in 38.011 seconds of wall-clock time (37.53 seconds user time and 0.09 seconds system time, at 98% CPU utilization). The assembly and verification measurements are reproducible from the saved JSON Lines log.

Artifact property	Value
Generation log	5,448 records; 9,381,911 bytes
Certificate	`maxsat-local-proof-v2`; 12,587,834 bytes
Tree	37,054 nodes; maximum depth 8
Refinement	60 nodes: 40 tail splits and 20 variable exposures
Strategies	7,128 roots
Decompositions	12,765 nodes: 8,402 semantic and 4,363 assignment
Leaves	15,612 calls and 1,489 imported-lemma applications
Largest local formula	13 clauses
Generation	38.011 seconds wall-clock time; 37.53 user, 0.09 system
Assembly	8.75 seconds; 129.2 MiB peak resident memory
Verification	30.72 seconds; 185.4 MiB peak resident memory

Table 3: Statistics for the generated certificate. The verifier reports `ACCEPT RELATIVE TO IMPORTED LEMMAS`.

The imported applications consist of 1,330 uses of `lemma3`, 119 of `lemma2`, 26 of `lemma4`, nine of `double_lemma3`, and five of `step5.1`. For exact artifact identification, the SHA-256 digests are

```
pre_certificate.jsonl  59656a240d59b0613f84b5d864a2e4bc...
proof_tree.json         01d18e63e7aa0a22a77670d8ea414bb8...
```

The full digests are recorded in `artifact_manifest.txt`; the imported-lemma applications are listed in `imported_lemmas.json`.

7 Conclusion

We developed a pipeline for turning a partially specified MaxSAT neighborhood into a checked local branching analysis. Structural refinements cover the possible neighborhoods, and strategy search combines dominance, grouping, reductions, and recursive analysis of residual formulas. The certificate makes these stages explicit while reducing their verification to coverage, local optimum identities, and exact recurrence arithmetic.

The $(3, 2)$-literal case demonstrates how this pipeline can meet the target 1.2872 and support a refinement of Xiao's analysis. The local soundness theorem and the global consequence identify the mathematical dependencies needed to use the generated tree. The same separation allows the search to be extended while keeping a common, inspectable verification interface.

References

[1] Mingyu Xiao. An exact MaxSAT algorithm: Further observations and further improvements. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, pages 1887–1893, 2022.

[2] Sergey S. Fedin and Alexander S. Kulikov. Automated proofs of upper bounds on the running time of splitting algorithms. In *IWPEC*, volume 3162 of LNCS, pages 248–259, 2004. doi:10.1007/978-3-540-28639-4_22.

[3] Alexander S. Kulikov. Automated generation of simplification rules for SAT and MAXSAT. In *SAT*, volume 3569 of LNCS, pages 430–436, 2005. doi:10.1007/11499107_35.

[4] Bjarke Skjernaa. *Exact Algorithms for Variants of Satisfiability and Colouring Problems*. PhD thesis, University of Aarhus, BRICS DS-04-5, 2004. Chapter 8. https://www.brics.dk/DS/04/5/BRICS-DS-04-5.pdf.

[5] Radovan Červený and Ondřej Suchý. Generating faster algorithms for d-Path Vertex Cover. In *WG*, pages 157–171, 2023. doi:10.1007/978-3-031-43380-1_12.

[6] Xuan-Zhao Gao, Yi-Jia Wang, Pan Zhang, and Jin-Guo Liu. Automated discovery of branching rules with optimal complexity for the Maximum Independent Set problem. arXiv:2412.07685, 2024. https://arxiv.org/abs/2412.07685.

[7] Nikhil Bansal and Venkatesh Raman. Upper bounds for MaxSat: Further improved. In *ISAAC*, pages 247–258, 1999.

[8] Jianer Chen and Iyad A. Kanj. Improved exact algorithms for MAX-SAT. *Discrete Applied Mathematics*, 142(1–3):17–27, 2004.

[9] Chao Xu, Wenjun Li, Yongjie Yang, Jianer Chen, and Jianxin Wang. Resolution and domination: An improved exact MaxSAT algorithm. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pages 1191–1197, 2019.

[10] Jianer Chen, Chao Xu, and Jianxin Wang. Dealing with 4-variables by resolution: An improved MaxSAT algorithm. *Theoretical Computer Science*, 670:33–44, 2017.

[11] Ivan Bliznets and Alexander Golovnev. A new algorithm for parameterized MAX-SAT. In *IPEC*, pages 37–48, 2012.

[12] Fedor V. Fomin and Dieter Kratsch. *Exact Exponential Algorithms*. Springer, 2010.

[13] Hannes Ihalainen, Andy Oertel, Yong Kiam Tan, Jeremias Berg, Matti Järvisalo, Magnus O. Myreen, and Jakob Nordström. Certified MaxSAT preprocessing. In *IJCAR*, volume 14739 of LNCS, pages 396–418, 2024. doi:10.1007/978-3-031-63498-7_24.